

الدليل الشامل للأكواد والصياغات البرمجية لمنصة ردود للذكاء الاصطناعي

Rudood AI Platform – Comprehensive Code, Algorithms & Syntax Deep-Dive Guide

الوثيقة: الدليل المرجعي الكامل للأكواد البرمجية والخوارزميات والصياغات (Code & Syntax Reference Manual)

الإصدار: Enterprise 2.0

إطار العمل واللغة: Laravel 11.x | PHP 8.4 / 8.2 | Node.js 20 | Vanilla JS

الهدف: تفكيك وشرح كل كلاس، دالة، معامل، استعلام قاعدة بيانات، ونمط صياغة في المنصة سطوراً بسطراً.

فهرس أقسام الدليل (Table of Contents)

1. معايير البنية الكودية والصياغة العامة (Code Architecture & Typing Standards)
 - 3.1 كود خدمة التوجيه الذكي AiService.php
 - 3.2 كود محرك البحث الدلالي والمتجهات RagService.php
 - 3.3 كود رسائل واتساب التفاعلية WhatsAppInteractiveService.php
 - 3.4 كود تكامل المتاجر واستدعاء الأدوات StoreIntegrationService.php
 - 3.5 كود تتبع التحويلات وحساب العائد ConversionTrackingService.php
 - 3.6 كود إحصائيات الإدارة والـ MRR AdminStatsService.php
4. ⚡ تشرح كود المهام الخلفية والأوامر (Background Jobs & Artisan Commands)
 - 4.1 معالج الرسائل غير المتزامن ProcessCustomerMessage.php
 - 4.2 ديمون الاستماع لتليجرام TelegramPollCommand.php
5. 🎮 تشرح كود المتحكمات وتدفق الطلبات (HTTP Controllers & Endpoints)
 - 5.1 متحكمات المصادقة والمستأجرين (AuthController , DashboardController , BotController)

- 5.2 Webhooks (ConversationController , والد , WebhookController , WidgetController)
- 5.3 متحكمات الإدارة العليا (AdminWorkspaceController , AdminDatabaseController , AdminSubscriberController)
- 6.  تشريح كود البرمجيات الوسيطة والأمان (Middleware & Security Guards)
- 7.  تشريح كود البث اللحظي ونصوص الواجهة (Real-Time WebSocket & Widget.js)
- 7.1 خادم الويب سوكت المستقل backend/websocket/server.js
- 7.2 ويدجت الموقع الخفيف backend/public/widget.js
- 8.  تشريح كود محرك الاختبارات الآلية (Automated Test Suite Runner)

1. معايير البنية الكودية والصياغة العامة (Code Architecture & Typing Standards)

تلتزم المنصة بأعلى معايير جودة الكود البرمجي في منظومة PHP و Laravel:

- **PHP 8.4 Strict Typing**: استخدام الـ Type Hinting الصارم على كافة المعاملات وقيم الإرجاع (, int , string , float , bool , array , ?string , JsonResponse , View , StreamedResponse).
- **Constructor Property Promotion**: صياغة حقن الاعتماديات الحديثة في الـ Constructors:

```
public function __construct(
    private Bot $bot,
    private ?RagService $ragService = null
) {}
```

- **Match Expressions**: استبدال جمل switch-case التقليدية بـ match لضمان أداء أسرع وقراءة أنظف للكود:

```
$apiKey = match ($provider) {
    'gemini'           => env('GEMINI_API_KEY'),
    'openai'           => env('OPENAI_API_KEY'),
    'anthropic'        => env('ANTHROPIC_API_KEY'),
    'openai_compatible' => $this->bot->api_key ? : env('OPENAI_API_KEY'),
    default             => env('GEMINI_API_KEY') ? : env('OPENAI_API_KEY'),
};
```

- **Safe Null Chaining & Coalescing**: استخدام مشغلات <? و ?? لمنع حدوث أخطاء الـ Null Pointer Exceptions.

2. تشرح كود نماذج البيانات والعلاقات (Eloquent) (Models & Database Schemas)

تحتوي المنصة على 19 نموذج Eloquent متكامل. نستعرض هنا صياغة أهم النماذج الأساسية وعلاقاتها:

1. نموذج الشركة / المستأجر Workspace.php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\HasOne;

class Workspace extends Model
{
    protected $fillable = [
        'company_name',
        'email',
        'phone',
        'status', // active, suspended, trial
        'plan_id', // free, starter, pro, enterprise
        'monthly_messages_quota', // e.g. 1000, 5000, 20000
        'messages_used_this_month', // counter
        'total_tokens_used', // cumulative tokens
        'webhook_secret', // HMAC verification token
    ];

    protected $casts = [
        'monthly_messages_quota' => 'integer',
        'messages_used_this_month' => 'integer',
        'total_tokens_used' => 'integer',
    ];

    // Relationships
    public function users(): HasMany { return $this->hasMany(User::class); }
    public function bots(): HasMany { return $this->hasMany(Bot::class); }
    public function defaultBot(): HasOne { return $this->hasOne(Bot::class)->where('is_
    public function conversations(): HasMany { return $this->hasMany(Conversation::clas
    public function channels(): HasMany { return $this->hasMany(Channel::class); }
```

```
public function autoRules(): HasMany { return $this->hasMany(AutoRule::class); }

// Quota Logic
public function hasRemainingQuota(): bool
{
    if ($this->monthly_messages_quota === 0) return true; // Unlimited
    return $this->messages_used_this_month < $this->monthly_messages_quota;
}

public function recordUsage(int $messageCount = 1, int $tokens = 0): void
{
    $this->increment('messages_used_this_month', $messageCount);
    if ($tokens > 0) {
        $this->increment('total_tokens_used', $tokens);
    }
}
}
```

2. نموذج تكوين البوت والذكاء الاصطناعي Bot.php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Bot extends Model
{
    protected $fillable = [
        'workspace_id',
        'name',
        'tone',                // formal, friendly, sales, custom
        'ai_provider',        // gemini, openai, anthropic, openai_compatible
        'model_type',         // e.g. gpt-4o-mini, gemini-1.5-flash
        'api_key',            // Custom encrypted store key
        'temperature',        // 0.0 to 1.0 (float)
        'max_tokens',         // e.g. 1000
        'system_prompt',      // Persona instructions
        'welcome_message',    // First response greeting
        'enable_auto_rules',  // boolean toggle
        'enable_rag',         // boolean toggle
        'is_active',          // boolean
    ];

    protected $casts = [
        'temperature'        => 'float',
        'max_tokens'         => 'integer',
        'enable_auto_rules'  => 'boolean',
        'enable_rag'         => 'boolean',
        'is_active'          => 'boolean',
    ];

    public function workspace(): BelongsTo { return $this->belongsTo(Workspace::class); }
    public function knowledgeBases(): HasMany { return $this->hasMany(KnowledgeBase::class); }
}
```

3. نموذج المحادثة وجلسات الشات Conversation.php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Conversation extends Model
{
    protected $fillable = [
        'workspace_id',
        'customer_id',
        'channel_id',
        'platform',           // whatsapp, telegram, instagram, web
        'status',             // open, in_progress, resolved, closed
        'is_bot_paused',      // true when human agent takes over
        'unread_count',       // unread messages for agent
        'context_summary',    // compressed long-thread history
        'sentiment',          // positive, neutral, frustrated, angry
        'is_escalated',       // urgent alarm flag
        'escalation_reason',   // e.g. "طلب مشرف + تهديد بشكوى"
        'converted_revenue',  // attributed sales SAR
        'converted_order_id', // linked MockOrder ID
        'csat_rating',        // 1 to 5 stars
        'csat_feedback',      // customer written review
        'last_message_at',    // timestamp for sorting inbox
    ];

    protected $casts = [
        'is_bot_paused'      => 'boolean',
        'is_escalated'       => 'boolean',
        'unread_count'       => 'integer',
        'converted_revenue'  => 'float',
        'csat_rating'        => 'integer',
        'last_message_at'    => 'datetime',
    ];

    public function messages(): HasMany { return $this->hasMany(Message::class)->orderBy
    public function customer(): BelongsTo { return $this->belongsTo(Customer::class); }
    public function workspace(): BelongsTo { return $this->belongsTo(Workspace::class); }

    public function isBotActive(): bool
```

```
{  
    return !$this->is_bot_paused;  
}  
  
public function markAsConverted(float $amount, int $orderId): void  
{  
    $this->update([  
        'converted_revenue' => $amount,  
        'converted_order_id' => $orderId,  
    ]);  
}  
}
```

4. نموذج المستندات والتدريب المعرفي KnowledgeBase.php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class KnowledgeBase extends Model
{
    protected $fillable = [
        'workspace_id',
        'bot_id',
        'file_name',
        'file_path',
        'document_text', // raw extracted text
        'chunks_json',   // JSON array of chunk strings/objects
    ];

    protected $casts = [
        'chunks_json' => 'array',
    ];

    // Accessor to normalize chunks
    public function getChunksAttribute(): array
    {
        $val = $this->chunks_json;
        if (is_array($val)) return $val;
        if (is_string($val)) return json_decode($val, true) ?? [];
        return [];
    }
}
```


3. تشرح كود الخدمات وخوارزميات الذكاء الاصطناعي (Domain Services & Algorithms)

3.1 كود خدمة التوجيه الذكي AiService.php

الملف backend/app/Services/AiService.php هو عصب الذكاء الاصطناعي للمنصة.

أ) دالة التوليد وتوجيه المزودين generateReply () :

```
public function generateReply(string $userMessage, string $context = '', array $history
{
    if (!empty($overrides)) {
        $this->overrides = array_merge($this->overrides, $overrides);
    }

    $this->lastError = null;
    $provider = $this->overrides['ai_provider'] ?? $this->bot->ai_provider ?: 'gemini';
    $apiKey = $this->overrides['api_key'] ?? $this->bot->api_key;

    if (!$apiKey) {
        $apiKey = match ($provider) {
            'gemini'          => env('GEMINI_API_KEY'),
            'openai'         => env('OPENAI_API_KEY'),
            'anthropic'      => env('ANTHROPIC_API_KEY'),
            'openai_compatible' => $this->bot->api_key ?: env('OPENAI_API_KEY'),
            default          => env('GEMINI_API_KEY') ?: env('OPENAI_API_KEY'),
        };
    }

    if (!$apiKey) {
        $this->lastError = "الخاص بمزود الذكاء الاصطناعي غير متوفر API مفتاح";
        return $this->getFallbackReply();
    }

    $normalizedHistory = $this->normalizeHistory($history);

    try {
        return match ($provider) {
            'openai'          => $this->callOpenAI($userMessage, $context, $apiKey, $
```

```
        'gemini'           => $this->callGemini($userMessage, $context, $apiKey, $
        'anthropic'       => $this->callAnthropic($userMessage, $context, $apiKey
        'openai_compatible' => $this->callOpenAiCompatible($userMessage, $context,
        default           => $this->callGemini($userMessage, $context, $apiKey, $
    };
} catch (\Exception $e) {
    $this->lastError = $e->getMessage();
    \Log::error('AI Service Error: ' . $e->getMessage());
    return $this->getFallbackReply();
}
}
```

ب) استدعاء Google Gemini API مع الذاكرة المحادثية:

```
private function callGemini(string $userMessage, string $context, string $apiKey, array
{
    $model = $this->overrides['model_type'] ?? $this->bot->model_type ?: 'gemini-1.5-fl
    $temp = $this->overrides['temperature'] ?? $this->bot->temperature ?? 0.7;

    $endpoint = "https://generativelanguage.googleapis.com/v1beta/models/{$model}:gener
    $systemPrompt = $this->buildSystemPrompt($context);

    $contents = [];

    // Append prior conversational turns
    foreach ($history as $turn) {
        $role = ($turn['role'] === 'assistant' || $turn['role'] === 'bot') ? 'model' :
        $contents[] = [
            'role' => $role,
            'parts' => [['text' => (string)($turn['content'] ?? '')]]
        ];
    }

    // Append current user message
    $contents[] = [
        'role' => 'user',
        'parts' => [['text' => $userMessage]]
    ];

    $payload = [
        'system_instruction' => [
            'parts' => [['text' => $systemPrompt]]
        ],
        'contents' => $contents,
        'generationConfig' => [
            'temperature' => (float) $temp,
            'maxOutputTokens' => (int) ($this->overrides['max_tokens'] ?? $this->bot->m
        ]
    ];

    $response = Http::timeout(30)->post($endpoint, $payload);

    if ($response->successful() && $text = $response->json('candidates.0.content.parts.
        return trim($text);
    }
```

```
$this->lastError = 'Gemini Error (' . $response->status() . '): ' . ($response->jso  
\Log::error($this->lastError);  
return $this->getFallbackReply();  
}
```

ج) خوارزمية تحليل المشاعر ورصد التصعيد analyzeSentimentAndUrgency () :

```
public function analyzeSentimentAndUrgency(string $message): array
{
    $lower = Str::lower($message);

    $angryKeywords = ['جارة', 'وزارة', 'شرطة', 'محامي', 'تافه', 'مسخرة', 'أغبياء', 'زفت'],
    $frustratedKeywords = ['ي', 'ليش', 'طولتو', 'ما تردون', 'خدمة سيئة', 'مشكلة', 'تعبت'],
    $positiveKeywords = ['نعم', 'ما قصرتمو', 'تسلم', 'يعطيكم العافية', 'أفضل متجر', 'جميل'];

    $isAngry = false;
    foreach ($angryKeywords as $word) {
        if (Str::contains($lower, $word)) { $isAngry = true; break; }
    }

    $isFrustrated = false;
    foreach ($frustratedKeywords as $word) {
        if (Str::contains($lower, $word)) { $isFrustrated = true; break; }
    }

    $isPositive = false;
    foreach ($positiveKeywords as $word) {
        if (Str::contains($lower, $word)) { $isPositive = true; break; }
    }

    if ($isAngry) {
        return [
            'sentiment' => 'angry',
            'is_escalated' => true,
            'urgency' => 'high',
            'reason' => 'تم رصد ألفاظ غضب شديد أو تهديد بشكوى رسمية من العميل',
        ];
    }

    if ($isFrustrated) {
        return [
            'sentiment' => 'frustrated',
            'is_escalated' => false,
            'urgency' => 'medium',
            'reason' => 'استياء من تأخر الطلب أو جودة الخدمة',
        ];
    }
}
```

```

if ($isPositive) {
    return [
        'sentiment' => 'positive',
        'is_escalated' => false,
        'urgency' => 'low',
        'reason' => 'انطباع إيجابي ورضا من العميل',
    ];
}

return [
    'sentiment' => 'neutral',
    'is_escalated' => false,
    'urgency' => 'low',
    'reason' => 'استفسار عام اعتيادي',
];
}

```

3.2 كود محرك البحث الدلالي والمتجهات RagService.php

الملف backend/app/Services/RagService.php ينفذ الحسابات الرياضية للمتجهات:

أ) حساب تشابه جيب التمام (Cosine Similarity) في PHP:

```
public function calculateCosineSimilarity(array $vecA, array $vecB): float
{
    if (empty($vecA) || empty($vecB)) return 0.0;

    $len = min(count($vecA), count($vecB));
    $dotProduct = 0.0;
    $normA = 0.0;
    $normB = 0.0;

    for ($i = 0; $i < $len; $i++) {
        $a = (float) $vecA[$i];
        $b = (float) $vecB[$i];
        $dotProduct += ($a * $b);
        $normA += ($a * $a);
        $normB += ($b * $b);
    }

    if ($normA <= 0.0 || $normB <= 0.0) {
        return 0.0;
    }

    return round($dotProduct / (sqrt($normA) * sqrt($normB)), 4);
}
```

ب) توليد المتجهات اللغوية خفيفة الوزن 64-بعداً generateVectorEmbedding () :

```
public function generateVectorEmbedding(string $text): array
{
    $text = Str::lower(trim($text));
    $dims = 64;
    $vector = array_fill(0, $dims, 0.0);

    // Deterministic hashing across character n-grams and tokens
    $tokens = array_filter(preg_split('/[\s,\.\?!.]+/u', $text));
    foreach ($tokens as $idx => $token) {
        $h = crc32($token);
        $dim = abs($h) % $dims;
        $weight = 1.0 + (mb_strlen($token) * 0.1);
        $vector[$dim] += $weight;

        // Bigram activation for contextual phrases
        if (isset($tokens[$idx + 1])) {
            $bigram = $token . '_' . $tokens[$idx + 1];
            $bgDim = abs(crc32($bigram)) % $dims;
            $vector[$bgDim] += ($weight * 1.2);
        }
    }

    // L2 Vector Normalization to unit length (length = 1.0)
    $norm = 0.0;
    foreach ($vector as $v) {
        $norm += ($v * $v);
    }
    $sqrtNorm = sqrt($norm);
    if ($sqrtNorm > 0) {
        foreach ($vector as $i => $v) {
            $vector[$i] = round($v / $sqrtNorm, 6);
        }
    }

    return $vector;
}
```


3.3 كود رسائل واتساب التفاعلية

WhatsAppInteractiveService.php

الملف : backend/app/Services/WhatsAppInteractiveService.php

```

public function buildButtonPayload(
    string $to,
    string $body,
    array $buttons,
    ?string $header = null,
    ?string $footer = 'ردود الذكاء الاصطناعي'
): array {
    $formattedButtons = [];
    $trimmedButtons = array_slice($buttons, 0, 3); // Meta allows max 3 buttons

    foreach ($trimmedButtons as $idx => $btn) {
        $id      = is_array($btn) ? ($btn['id'] ?? 'btn_' . ($idx + 1)) : 'btn_' . ($idx
        $title    = is_array($btn) ? ($btn['title'] ?? (string)$btn) : (string)$btn;
        $title    = mb_substr(trim($title), 0, 20); // Title max 20 chars

        $formattedButtons[] = [
            'type' => 'reply',
            'reply' => [
                'id'      => (string) $id,
                'title' => $title,
            ]
        ];
    }

    $interactive = [
        'type'      => 'button',
        'body'      => ['text' => $body],
        'action' => ['buttons' => $formattedButtons]
    ];

    if (!empty($header)) $interactive['header'] = ['type' => 'text', 'text' => mb_substr
    if (!empty($footer)) $interactive['footer'] = ['text' => mb_substr($footer, 0, 60)]

    return [
        'messaging_product' => 'whatsapp',
        'recipient_type'    => 'individual',
        'to'                 => ltrim($to, '+'),
        'type'               => 'interactive',
        'interactive'        => $interactive,
    ];
}

```

3.4 كود تكامل المتاجر واستدعاء الأدوات

StoreIntegrationService.php

الملف : backend/app/Services/StoreIntegrationService.php

```
public function checkOrderStatus(string $query, ?int $workspaceId = null): array
{
    // Clean order number: removes '#' or 'طلب' or 'No.'
    $cleanNumber = trim(str_replace(['#', 'No.', 'no.', 'order', 'طلب'], '', $query));

    $orderQuery = MockOrder::query();
    if ($workspaceId) {
        $orderQuery->where(function($q) use ($workspaceId) {
            $q->where('workspace_id', $workspaceId)->orWhereNull('workspace_id');
        });
    }

    $order = $orderQuery->where('order_number', $cleanNumber)
        ->orWhere('order_number', 'like', "%{$cleanNumber}%")
        ->orWhere('customer_phone', 'like', "%{$cleanNumber}%")
        ->first();

    if ($order) {
        return [
            'found' => true,
            'order_number' => $order->order_number,
            'status' => $order->status_label,
            'courier' => $order->courier ?: 'أرامكس (Aramex)',
            'tracking_number' => $order->tracking_number ?: 'TRK-' . rand(100000, 999999),
            'total_amount' => $order->total_amount . ' ر.س',
            'estimated_delivery' => $order->estimated_delivery ?: 'خلال 24-48 ساعة عمل',
            'tracking_url' => "https://track.rudood.ai/" . ($order->tracking_number),
        ];
    }

    return [
        'found' => false,
        'searched_number' => $cleanNumber,
        'message' => "حوال لمساعدتك {$cleanNumber} لم يتم العثور على طلب بالرقم",
    ];
}
```

3.5 كود تتبع التحويلات وحساب العائد

ConversionTrackingService.php

الملف : backend/app/Services/ConversionTrackingService.php

```

public function attributeOrderToConversation(
    MockOrder $order,
    ?Conversation $conversation = null,
    string $type = 'catalog_order',
    float $confidence = 1.00
): array {
    // Lookup matching conversation within 72-hour window by phone
    if (!$conversation) {
        $convQuery = Conversation::query();
        if ($order->workspace_id) {
            $convQuery->where('workspace_id', $order->workspace_id);
        }

        $cleanPhone = preg_replace('/[^0-9]/', '', (string)$order->customer_phone);
        $lastDigits = strlen($cleanPhone) >= 7 ? substr($cleanPhone, -7) : $cleanPhone;

        if (!empty($lastDigits)) {
            $convQuery->whereHas('customer', function ($q) use ($lastDigits) {
                $q->where('phone', 'like', "%{$lastDigits}%");
            });
        }

        $conversation = $convQuery->where('created_at', '>=', now()->subHours(72))
            ->latest()
            ->first();
    }

    if ($conversation) {
        $order->update([
            'conversation_id' => $conversation->id,
            'is_attributed_to_bot' => true,
            'attribution_type' => $type,
            'attribution_confidence' => $confidence,
        ]);

        $conversation->markAsConverted((float)$order->total_amount, $order->id);

        return [
            'attributed' => true,
            'order_id' => $order->id,
            'order_number' => $order->order_number,
            'conversation_id' => $conversation->id,
            'revenue' => (float)$order->total_amount,
        ];
    }
}

```

```
    ];  
}  
  
return ['attributed' => false, 'order_id' => $order->id];  
}
```

4. ⚡ تشرح كود المهام الخلفية والأوامر (Background) (Jobs & Artisan Commands)

4.1 معالجة الرسائل غير المتزامنة ProcessCustomerMessage.php

الملف : backend/app/Jobs/ProcessCustomerMessage.php

```

namespace App\Jobs;

use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Queue\Queueable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use App\Models\Message, Conversation, Bot, Channel, AiDecisionLog;
use App\Services\AiService, RagService, WhatsAppInteractiveService;
use Illuminate\Support\Facades\Redis, Http;

class ProcessCustomerMessage implements ShouldQueue
{
    use Queueable, InteractsWithQueue, SerializesModels;

    public int $tries = 3;
    public int $backoff = 30;

    public function __construct(
        private int $conversationId,
        private int $messageId
    ) {}

    public function handle(?RagService $ragService = null): void
    {
        $ragService = $ragService ?? app(RagService::class);
        $startTime = microtime(true);

        $conversation = Conversation::with('customer', 'workspace')->find($this->conversationId);
        if (!$conversation || !$conversation->isBotActive()) return;

        $workspace = $conversation->workspace;
        if ($workspace && !$workspace->hasRemainingQuota()) return;

        $bot = Bot::where('workspace_id', $conversation->workspace_id)->where('is_active', 1)->first();
        if (!$bot) return;

        $customerMessage = Message::find($this->messageId);
        if (!$customerMessage) return;

        // 1. Analyze Sentiment
        $aiService = new AiService($bot);
        try {
            $sentiment = $aiService->analyzeSentimentAndUrgency($customerMessage->content);
        } catch (\Exception $e) {
            // Handle exception
        }
    }
}

```

```

        $conversation->update([
            'sentiment' => $sentiment['sentiment'],
            'is_escalated' => $sentiment['is_escalated'] || $conversation->is_escalated,
            'escalation_reason' => $sentiment['reason'] ?? $conversation->escalation_reason
        ]);
    } catch (\Throwable $e) {}

// 2. Decision Pipeline (Auto-Rules -> Tool Calling -> RAG + LLM)
$trigger = 'ai_api';
$matchedKeywords = null;
$context = null;

$enableAutoRules = ($bot->enable_auto_rules ?? true) !== false;
$enableRag = ($bot->enable_rag ?? true) !== false;

$ruleMatch = $enableAutoRules ? $ragService->checkAutoRules($conversation->work

if ($ruleMatch !== null) {
    $trigger = 'auto_rule';
    $replyText = $ruleMatch['reply'];
    $matchedKeywords = is_array($ruleMatch['keywords']) ? implode(', ', $ruleMa
} else {
    $toolResult = $aiService->executeToolCalls($customerMessage->content, $conv
    if ($toolResult !== null) {
        $trigger = 'ai_tool:' . $toolResult['tool'];
        $replyText = $toolResult['reply'];
    } else {
        $history = Message::where('conversation_id', $conversation->id)->where(
        if ($enableRag) {
            $ragRes = $ragService->retrieveRelevantChunks($bot->id, $customerM
            $context = $ragRes['context'];
        }
        $replyText = $aiService->generateReply($customerMessage->content, $cont
    }
}

// 3. Save Bot Message
$botMessage = Message::create([
    'conversation_id' => $conversation->id,
    'sender_type' => 'bot',
    'content' => $replyText,
]);
$conversation->touch();

```



```

if ($workspace) $workspace->recordUsage(1, mb_strlen($replyText));

// 4. Record Decision Telemetry Log
$durationMs = (int) max(1, round((microtime(true) - $startTime) * 1000));
AiDecisionLog::create([
    'conversation_id' => $conversation->id,
    'message_id'      => $botMessage->id,
    'trigger'         => $trigger,
    'matched_keywords' => $matchedKeywords,
    'ai_provider'     => $bot->ai_provider ?: 'gemini',
    'customer_message' => $customerMessage->content,
    'bot_reply'       => $replyText,
    'response_time_ms' => $durationMs,
]);

// 5. Publish to Redis -> WebSocket
try {
    Redis::publish('rudood_chat_channel', json_encode([
        'conversation_id' => $conversation->id,
        'workspace_id'    => $conversation->workspace_id,
        'sender_type'     => 'bot',
        'content'         => $botMessage->content,
        'time'            => $botMessage->created_at->format('H:i'),
        'message_id'      => $botMessage->id,
    ]));
} catch (\Throwable $e) {}

// 6. Dispatch Outgoing API Message
$this->dispatchOutgoingChannelMessage($conversation, $botMessage);
}
}

```

5. تشرح كود المتحكمات وتدفق الطلبات (HTTP) (Controllers & Endpoints)

5.1 متحكمات المصادقة والمستأجرين

التسجيل الذري للشركة: AuthController::register () —

```
public function register(Request $request): RedirectResponse
{
    $validated = $request->validate([
        'name'          => 'required|string|max:255',
        'company_name' => 'required|string|max:255',
        'email'         => 'required|email|unique:users,email',
        'password'      => 'required|string|min:8|confirmed',
    ]);

    DB::transaction(function () use ($validated) {
        $workspace = Workspace::create([
            'company_name' => $validated['company_name'],
            'email'        => $validated['email'],
            'status'       => 'active',
            'plan_id'      => 'starter',
            'monthly_messages_quota' => 1000,
        ]);

        $user = User::create([
            'workspace_id' => $workspace->id,
            'name'         => $validated['name'],
            'email'        => $validated['email'],
            'password'     => Hash::make($validated['password']),
            'role'         => 'owner',
        ]);

        Bot::create([
            'workspace_id' => $workspace->id,
            'name'         => 'مساعد' . $workspace->company_name,
            'tone'         => 'friendly',
            'ai_provider'  => 'gemini',
            'model_type'   => 'gemini-1.5-flash',
        ]);
    });
}
```

```
'temperature'      => 0.7,  
'system_prompt'    => 'أنت مساعد خدمة عملاء ذكي لمتجر' . $workspace->compa  
'welcome_message'  => 'أهلاً بك في متجرنا! كيف أستطيع خدمتك اليوم؟',  
'is_active'        => true,  
'enable_auto_rules' => true,  
'enable_rag'       => true,  
]);
```

```
Auth::login($user);  
});
```

```
return redirect('/dashboard')->with('success', 'نشأ متجرك بنجاح وتفعيل البوت الذكي'  
}
```

5.2 متحكمات المحادثات والـ Webhooks

WebhookController::handleWhatsApp () — معالجة رسائل واتساب:

```
public function handleWhatsApp(Request $request): JsonResponse
{
    $payload = $request->all();
    $entry    = $payload['entry'][0]['changes'][0]['value'] ?? null;

    if (!$entry || !isset($entry['messages'][0])) {
        return response()->json(['status' => 'ignored'], 200);
    }

    $messageData = $entry['messages'][0];
    $fromPhone    = $messageData['from'];
    $text         = $messageData['text']['body'] ?? ($messageData['interactive']['button

// Resolve Customer & Conversation
$customer = Customer::firstOrCreate(
    ['phone' => $fromPhone],
    ['name' => $entry['contacts'][0]['profile']['name'] ?? 'عميل واتساب', 'platform

);

$conversation = Conversation::firstOrCreate(
    ['customer_id' => $customer->id, 'platform' => 'whatsapp', 'status' => 'open'],
    ['workspace_id' => 1, 'last_message_at' => now()]
);

$customerMessage = Message::create([
    'conversation_id' => $conversation->id,
    'sender_type'     => 'customer',
    'content'         => $text,
]);

// Dispatch background job
ProcessCustomerMessage::dispatch($conversation->id, $customerMessage->id);

return response()->json(['status' => 'received'], 200);
}
```

6. تشرح كود البرمجيات الوسيطة والأمان (Middleware & Security Guards)

1. وسيط وضع الصيانة CheckMaintenanceMode.php

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use App\Models\SystemSetting;

class CheckMaintenanceMode
{
    public function handle(Request $request, Closure $next)
    {
        $isMaintenance = SystemSetting::get('maintenance_mode', false);

        if ($isMaintenance) {
            $user = auth()->user();

            // Super Admins bypass maintenance mode
            if ($user && $user->isSuperAdmin()) {
                return $next($request);
            }

            // Exclude public landing page and maintenance route itself
            if ($request->is('/') || $request->is('index') || $request->is('maintenance'))
                return $next($request);
        }

        return redirect()->route('maintenance');
    }

    return $next($request);
}
```

7. تشرح كود البث اللحظي ونصوص الواجهة (Real-Time WebSocket & Widget.js)

7.1 خادم الويب سوكت المستقل backend/websocket/server.js

```
const express = require('express');
const http    = require('http');
const { Server } = require('socket.io');
const Redis    = require('ioredis');

const app      = express();
const server   = http.createServer(app);
const io       = new Server(server, {
  cors: { origin: process.env.CORS_ORIGIN || '*', methods: ['GET', 'POST'] }
});

const redisSub = new Redis({
  host: process.env.REDIS_HOST || '127.0.0.1',
  port: parseInt(process.env.REDIS_PORT || '6379', 10),
});

// Subscribe to Laravel Chat Channel
redisSub.subscribe('rudood_chat_channel', () => {
  console.log('[Redis] Subscribed to rudood_chat_channel');
});

// Broadcast on message received from Laravel
redisSub.on('message', (channel, message) => {
  try {
    const data = JSON.parse(message);
    const workspaceRoom = `workspace_${data.workspace_id}`;

    // Emit to workspace agent inbox
    io.to(workspaceRoom).emit('new_message', data);
    io.to(workspaceRoom).emit('conversation_updated', {
      conversation_id: data.conversation_id,
      last_message:   data.content,
      time:           data.time,
    });
  } catch (e) {
```

```
    console.error('[Redis] Parse error:', e.message);
  }
});

io.on('connection', (socket) => {
  socket.on('join_workspace', (workspace_id) => {
    socket.join(`workspace_${workspace_id}`);
  });
});

server.listen(process.env.PORT || 3000, () => {
  console.log('WebSocket Server running on port 3000');
});
```

7.2 ويدجت الموقع الخفيف backend/public/widget.js

```
(function () {  
  if (window.__RUDOOD_WIDGET_LOADED__) return;  
  window.__RUDOOD_WIDGET_LOADED__ = true;  
  
  const currentScript = document.currentScript || document.querySelector('script[src*="']  
  const workspaceId = currentScript ? currentScript.getAttribute('data-workspace') :  
  const apiBase = currentScript ? currentScript.getAttribute('data-api') || window.  
  
  // Render Widget Bubble & Container  
  const container = document.createElement('div');  
  container.id = 'rudood-widget-container';  
  container.innerHTML = `  
  
    <div>  
      <div>مساعد المتجر الذكي</div>  
  
      <button id="rudood-close-btn">&times;</button>  
  
      <div>  
        <input type="text" id="rudood-user-input" placeholder="اكتب استفسارك هنا..."/>  
        <button id="rudood-send-btn">إرسال</button>  
      </div>  
    </div>  
  `;  
  document.body.appendChild(container);  
  
  // Event Listeners for message dispatch  
  const launcher = document.getElementById('rudood-widget-launcher');  
  const win = document.getElementById('rudood-widget-window');  
  const input = document.getElementById('rudood-user-input');  
  const sendBtn = document.getElementById('rudood-send-btn');  
  const msgBox = document.getElementById('rudood-messages');  
  
  launcher.addEventListener('click', () => {  
    win.style.display = win.style.display === 'none' ? 'flex' : 'none';  
  });
```



```

async function sendMessage() {
  const text = input.value.trim();
  if (!text) return;
  input.value = '';

  // Append user message to UI
  msgBox.innerHTML += `${text}`;
  msgBox.scrollTop = msgBox.scrollHeight;

  // Post to Rudood API
  try {
    const res = await fetch(`${apiBase}/api/widget/message`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ workspace_id: workspaceId, message: text })
    });
    const data = await res.json();
    if (data.reply) {
      msgBox.innerHTML += `${data.reply}`;
      msgBox.scrollTop = msgBox.scrollHeight;
    }
  } catch (err) {
    console.error('Widget error:', err);
  }
}

sendBtn.addEventListener('click', sendMessage);
input.addEventListener('keypress', (e) => { if (e.key === 'Enter') sendMessage(); });
})();

```

8. تشرح كود محرك الاختبارات الآلية (Automated) (Test Suite Runner)

الملف backend/tests_suite_runner.php يوفر إطار عمل اختبار مخصص وسريع بدون تحميل مكتبات خارجية معقدة:

```

class TestRunner
{
    private int $passed = 0;
    private int $failed = 0;

    public function assert(bool $condition, string $testName): void
    {
        if ($condition) {
            $this->passed++;
            echo " \033[32m✓ PASS:\033[0m {$testName}\n";
        } else {
            $this->failed++;
            echo " \033[31m✗ FAIL:\033[0m {$testName}\n";
        }
    }

    public function runSuite(string $title, callable $tests): void
    {
        echo "\n\033[1;34m📦 Suite: {$title}\033[0m\n";
        $tests($this);
    }

    public function report(): void
    {
        $total = $this->passed + $this->failed;
        $rate = $total > 0 ? round(($this->passed / $total) * 100, 1) : 0;
        echo "\n===== \n";
        echo "Total: {$total} | Passed: {$this->passed} | Failed: {$this->failed} | Suc
        echo "===== \n";
    }
}

```

 تم إعداد وتدقيق هذا الدليل البرمجي الشامل بواسطة كبير مهندسي البرمجيات ومدير المعمارية التقنية للمنصة
 منصة ردود للذكاء الاصطناعي — مرجع الأكواد والخوارزميات الدقيقة